

AdminUnitTesting

For the Administrator unit tests, a `@BeforeEach` setup method was implemented to initialize all required dependencies. These included instances of `Manager`, `Cashier`, and `Sector`, which were passed to the `Administrator` constructor to simulate a realistic system state before each test execution. This ensured that every test was executed in a controlled and repeatable environment.

Tested Methods and Results

The following methods were successfully unit tested:

`getManagers()`, `getCashiers()`, `getSectors()`

These methods were tested to verify that they correctly return defensive copies of the internal lists. The tests confirmed that the expected number of elements was returned after initialization.

`addManager(Manager m)`, `addCashier(Cashier c)`

These methods correctly added new employees to the `Administrator`'s internal lists. Unit tests verified that the list sizes increased as expected after insertion.

`removeManager(String managerId)`, `removeCashier(String cashierId)`

The tests confirmed that employees were correctly removed based on their identifiers and that the corresponding lists were updated accordingly.

`modifyManagerId(String newId, String managerId)`, `modifyCashierId(String newId, String cashierId)`

These methods were tested to ensure that employee identifiers were correctly updated when valid IDs were provided.

`givePermission(String employeeId)`

This method was tested to verify that permissions were successfully restored for employees. The test case for granting permission to a manager passed successfully.

`employeeTask()`

The method returned a valid description of the administrator's responsibilities, confirming correct polymorphic behavior inherited from the `Employee` class.

`login(String username, String password)`

Both positive and negative authentication scenarios were tested. The method correctly validated credentials and returned the expected boolean result.

`getAdminId()`

The administrator ID getter method returned the expected identifier.

All of the above tests passed successfully, confirming the correct behavior of the Administrator's core business logic.

`revokePermission(String employeeId)`

The method `revokePermission(String employeeId)` was also tested; however, the corresponding unit test did not pass consistently. This failure is not caused by incorrect test logic, but by a design limitation in the model class.

The Administrator class directly performs file serialization inside its business logic through the `update()` method, which is invoked during permission changes. Since the project does not implement a dedicated DAO layer, persistence logic is tightly coupled with domain logic. As a result, unit tests are no longer fully isolated and become dependent on file system state.

Bill Unit Testing

For the Bill unit tests, a `@BeforeEach` setup method was implemented to initialize all required dependencies before every test execution. This included creating instances of `Cashier` and multiple `Item` objects, along with their corresponding quantities. These objects were passed to the `Bill` constructor to simulate a realistic billing scenario.

This setup ensured that each test was executed in a controlled, repeatable, and independent environment.

Tested Methods and Results

The following methods of the `Bill` class were successfully unit tested:

`getBillNumber()`

This method was tested to verify that a valid bill number is generated upon bill creation. The test confirmed that the returned value was non-negative.

`getCreated()`

The test verified that the bill creation date is properly initialized and not null.

`getCreatedBy()`

This method was tested to ensure that the cashier who created the bill is correctly assigned and returned.

`getTotalAmountOfBill()`

The total bill amount calculation was tested using predefined items and quantities. The test confirmed that the method correctly calculates the total based on item prices and quantities.

`addItem(Item item, int quantity)`

This method was tested to ensure that new items can be added to the bill and that the total amount updates accordingly.

An additional test case verified that invalid quantities (negative values) do not affect the bill total.

`isProductDiscounted(String productName)`

This method was tested for both existing and non-existing products. The tests confirmed that the method correctly returns false when no discount is applied or when the product is not found.

`applyDiscount(int percentage)`

Multiple test cases were implemented to validate discount application:

Valid discount percentages correctly reduced the total bill amount

A zero discount returned the original total

Invalid discount values (negative or greater than 100) correctly threw an `IllegalArgumentException`

`assignCashier(Cashier cashier)`

This method was tested to confirm that a new cashier can be assigned to an existing bill and that the change is reflected correctly.

`getFile()`

The test verified that a file associated with the bill is created successfully and exists in the file system.

`toString()`

The `toString()` method was tested to ensure it returns a non-null string containing key bill information, including the total amount.

Test Outcome

All unit tests for the Bill class passed successfully.

The results confirm that the Bill class behaves correctly under normal and boundary conditions, and that its core business logic is reliable and well-tested.

Cashier Unit Testing

For the Cashier unit tests, a @BeforeEach setup method was implemented to initialize all required dependencies prior to each test execution. This included creating instances of Item objects and initializing a Cashier with predefined personal information, credentials, and an initial list of items.

This setup ensured that each test was executed in a controlled, consistent, and repeatable environment.

Tested Methods and Results

The following methods of the Cashier class were successfully unit tested:

`getCashierId()`

This method was tested to verify that the cashier identifier is correctly returned after initialization.

`hasPermissionToWork()` / `setPermission(boolean)`

These methods were tested to confirm that a cashier has permission to work by default and that permission can be revoked or restored correctly.

`addAmount(double amount)`

This method was tested to ensure that amounts are correctly added to both the daily total and the total accumulated earnings.

`getItem()`

The test confirmed that the cashier initially holds the correct number of items assigned at construction.

`addItem(Item item)`

This method was tested to verify that new items can be added successfully to the cashier's item list.

`createBillOneItem(Item item, int quantity)`

This method was tested to confirm that a bill can be successfully created for a single item and that the cashier's bill count and daily total are updated accordingly.

An additional test verified that invalid input (null item) correctly throws an `IllegalArgumentException`.

`createBill(List<Item> items, List<Integer> quantities)`

This method was tested to ensure that bills can be created using multiple items and quantities.

Invalid input scenarios (null lists) were also tested and correctly resulted in an exception.

`resetTotalForDay()`

This method was tested to verify that the daily total amount is correctly reset to zero.

`startShift()`

The test confirmed that starting a new shift resets the daily total amount.

`endShift()`

This method was tested to ensure that the cashier's working day counter is incremented correctly at the end of a shift.

`getbills()`

The test verified that the list of bills associated with the cashier is correctly initialized and not null.

`login(String username, String password)`

Both valid and invalid login scenarios were tested. The method correctly authenticated valid credentials and rejected incorrect ones.

`EmployeeTask()`

This method was tested to confirm that a valid description of the cashier's responsibilities is returned, demonstrating correct polymorphic behavior inherited from the `Employee` class.

`toString()`

The `toString()` method was tested to ensure it returns a non-null string representation of the cashier object.

Test Outcome

All unit tests for the `Cashier` class passed successfully.

The results confirm that the `Cashier` class functions correctly under normal and edge-case conditions, and that its core responsibilities related to billing, authentication, and shift management are reliably implemented and tested.

Employee Unit Testing

For the Employee unit tests, a `@BeforeEach` setup method was implemented to initialize a concrete test instance of the abstract Employee class. Since Employee is abstract, a static inner class (TestEmployee) was created to provide implementations for the abstract methods.

This approach allowed the core functionality of the Employee class to be tested in isolation while maintaining a controlled and repeatable test environment.

Tested Methods and Results

The following methods of the Employee class were successfully unit tested:

`getEmployeeId() / setEmployeeId(String)`

These methods were tested to verify that the employee identifier is correctly returned and updated when a new valid ID is assigned.

`getRole() / setRole(String)`

The tests confirmed that the employee role is properly retrieved and can be modified as expected.

`getSalary() / setSalary(double)`

These methods were tested to ensure that the employee salary is correctly stored, retrieved, and updated.

`employeeTask()`

This method was tested to confirm that a valid task description is returned, demonstrating correct polymorphic behavior through method overriding.

`EmployeeTask()`

The capitalized version of the task method was tested to ensure that it returns the expected task description.

`toString()`

The `toString()` method was tested to verify that it returns a non-null string containing key employee information, including the employee ID, role, and salary.

Test Outcome

All unit tests for the Employee class passed successfully.

The results confirm that the Employee class correctly manages core employee attributes and supports polymorphic behavior, providing a reliable foundation for derived classes such as Cashier, Manager, and Administrator.

Inventory Unit Testing

For the Inventory unit tests, a @BeforeEach setup method was implemented to initialize the inventory with a predefined list of Item objects. This ensured that each test was executed with a known and consistent initial state, allowing accurate verification of inventory behavior.

Tested Methods and Results

The following methods of the Inventory class were successfully unit tested:

getNumberOfItems()

This method was tested to verify that the inventory correctly reports the number of items after initialization and after adding new items.

getSoldQuantityTotal()

The test confirmed that the total sold quantity is correctly initialized to zero.

getItems()

This method was tested to ensure that the returned list of items is unmodifiable. The test verified that attempting to modify the list throws an UnsupportedOperationException, confirming proper encapsulation and defensive programming.

addNewItem(Item item)

This method was tested to confirm that new items are added correctly to the inventory. An additional test verified that passing a null item does not modify the inventory state.

giveReportAfterPeriod(Date date)

This method was tested using dates both before and after the inventory creation time. The tests confirmed that the method correctly returns zero when no sales data exists for the specified period.

Test Outcome

All unit tests for the Inventory class passed successfully.

The results confirm that the Inventory class correctly manages item storage, enforces encapsulation of internal collections, and handles reporting logic reliably under boundary conditions.

Item Unit Testing

For the Item unit tests, a @BeforeEach setup method was implemented to initialize an Item instance with predefined attributes, including pricing, quantity, and supplier reference. This ensured that each test was executed with a known and consistent initial state, allowing accurate validation of stock management and pricing behavior.

Tested Methods and Results

The following methods of the Item class were successfully unit tested:

increaseStock(int quantity)

This method was tested to verify that increasing stock with a valid quantity correctly updates the item's available quantity and confirms that the item is not out of stock.

An additional test verified that invalid input (zero quantity) correctly throws an IllegalArgumentException.

decreaseStock(int quantity)

This method was tested to ensure that decreasing stock with a valid quantity correctly reduces the available quantity and updates the number of sold items.

Invalid and negative quantities were also tested and correctly resulted in an exception.

isItemOutOfStock()

This method was tested to confirm that the item is correctly marked as out of stock when the available quantity reaches zero.

applyDiscount(int percentage)

This method was tested to ensure that valid discount percentages correctly reduce the selling price.

Invalid discount values were tested and correctly caused an IllegalArgumentException.

itemsWithTheSamePrice(List<Item> items, double price)

This static method was tested to verify that it correctly counts the number of items with the same selling price within a given list.

`getPurchaseDate()`

This method was tested to confirm that it returns a defensive copy of the purchase date, ensuring that the internal state of the object cannot be modified externally.

`setQuantity(int quantity)`

This method was tested to verify that the item quantity is correctly updated when a new valid value is provided.

Test Outcome

All unit tests for the Item class passed successfully.

The results confirm that the Item class correctly manages stock levels, pricing logic, discount application, and data encapsulation, ensuring reliable behavior within the inventory and billing subsystems.

Manager Unit Testing

For the Manager unit tests, a `@BeforeEach` setup method was implemented to initialize all required dependencies before each test execution. This included instances of Cashier, Supplier, Item, and Sector, along with the creation of necessary directory structures to support file-based operations.

The Manager object was initialized with predefined lists of sectors, cashiers, suppliers, and items in order to simulate a realistic management environment.

Tested Methods and Results

The following methods of the Manager class were successfully unit tested:

`addCashier(Cashier cashier)`

This method was tested to verify that new cashiers are correctly added to the manager's internal list.

`addSector(Sector sector)`

The test confirmed that new sectors can be added successfully and that the sector list size increases accordingly.

`removeSector(String sectorName)`

This method was tested to ensure that a sector is correctly removed based on its name.

isAnySectorEmpty()

The method correctly returned a sector when at least one sector satisfied the empty-sector condition.

increaseStock(Item item, int quantity)

This method was tested to confirm that item stock levels are increased correctly.

addSupplier(Supplier supplier)

The test verified that new suppliers are correctly added to the manager's supplier list.

login(String username, String password)

Both valid and invalid authentication scenarios were tested. The method correctly validated credentials and returned the expected boolean result.

removeItem(String itemName)

This method was tested to ensure that items are correctly removed from the manager's inventory list.

setPermissionToWork(boolean)

The test confirmed that the manager's permission status can be updated correctly.

addTotalAmountSpent(double amount)

This method was tested to verify that the total amount spent by the manager is updated correctly.

employeeTask()

The method returned a valid description of the manager's responsibilities, confirming correct polymorphic behavior inherited from the Employee class.

Tests with Inconsistent Results

The following tests did not pass consistently:

removeCashier(String cashierId)

isOneSectorEmpty(String sectorName)

rateCashier(String cashierId)

These test failures are not caused by incorrect test logic, but by a design limitation in the model layer.

The Manager class performs file-based persistence operations directly within its business logic (e.g., during removal or evaluation operations). Since the project does not implement

a dedicated DAO (Data Access Object) layer, persistence logic is tightly coupled with domain logic.

As a result, unit tests are no longer fully isolated and become dependent on file system state, leading to inconsistent behavior during test execution.

Test Outcome

Most unit tests for the Manager class passed successfully, confirming the correctness of its core business logic.

The failing tests highlight an architectural limitation rather than functional errors, emphasizing the need for separation of concerns through a DAO or repository layer to improve testability and maintainability.

Sector Unit Testing

For the Sector unit tests, a @BeforeEach setup method was implemented to initialize a sector with a predefined list of items, their corresponding quantities, and associated suppliers. This ensured that each test was executed with a consistent and well-defined initial state, allowing accurate validation of sector-level inventory behavior.

Tested Methods and Results

The following methods of the Sector class were successfully unit tested:

isSectorEmpty()

This method was tested to verify whether the sector correctly reports its empty or non-empty state based on item quantities.

isItemOutOfStock(Item item)

The method was tested for both in-stock and out-of-stock items, confirming correct identification of stock status.

firstOutOfStockItem()

This method was tested to ensure that the first out-of-stock item within the sector is correctly identified and returned.

addNewItem(Item item)

The test confirmed that new items can be added to the sector and that the total number of items is updated correctly.

`deleteItem(Item item)`

This method was tested to verify that an existing item is correctly removed from the sector and that the total quantity is updated accordingly.

`updateItemQuantity(Item item, int quantity)`

The test verified that updating an item's quantity correctly modifies both the internal quantity list and the total number of items.

`getSectorName()`

This method was tested to ensure that the sector name is correctly returned.

Test Outcome

All unit tests for the Sector class passed successfully.

The results confirm that the Sector class correctly manages item quantities, stock status, and sector-level inventory operations.

Supplier Unit Testing

For the Supplier unit tests, individual test cases were implemented to validate the correct initialization and behavior of the Supplier class. The tests focused on verifying attribute assignment and basic object behavior.

Tested Methods and Results

The following methods of the Supplier class were successfully unit tested:

`Supplier(boolean isSupplierForStore, String name)`

The constructor was tested to verify that supplier attributes are correctly initialized.

`getName() / setName(String)`

These methods were tested to confirm that the supplier name is correctly retrieved and updated.

`getIsSupplierForStore() / setIsSupplierForStore(boolean)`

The tests verified that the supplier's association with the store is correctly managed.

`toString()`

The `toString()` method was tested to ensure that it returns a string representation containing the supplier name.

Test Outcome

All unit tests for the Supplier class passed successfully.

The results confirm that the Supplier class behaves correctly and reliably as a supporting entity within the system.

User Unit Testing

For the User unit tests, a @BeforeEach setup method was implemented to initialize a concrete test instance of the abstract User class. Since User is abstract, a static inner class (TestUser) was created to provide an implementation of the abstract login method.

This approach allowed the core functionality of the User class to be tested in isolation while maintaining a controlled and repeatable test environment.

Tested Methods and Results

The following methods of the User class were successfully unit tested:

Getter Methods

The getter methods were tested to verify that all user attributes (name, surname, phone number, address, username, and password) are correctly returned after initialization.

Setter Methods

These methods were tested to confirm that user attributes can be updated correctly, including name, surname, phone number, and address.

changePass(String oldPass, String newPass, String confirmPass)

This method was tested under multiple scenarios:

Successful password change when the old password is correct and the confirmation matches

Failure when the old password is incorrect

Failure when the new password and confirmation do not match

The tests confirmed that the password is only updated when all conditions are satisfied.

login(String username, String password)

Both valid and invalid login scenarios were tested. The method correctly authenticated valid credentials and rejected incorrect ones.

toString()

The toString() method was tested to ensure that it returns a non-null string representation of the user object.

Test Outcome

All unit tests for the User class passed successfully.

The results confirm that the User class correctly manages user information, authentication logic, and password updates, providing a reliable base for derived classes such as Employee.